

Foundations of Reinforcement Learning and Interactive Decision Making

DYLAN J. FOSTER AND ALEXANDER RAKHLIN

Last Updated: December 2023

These lecture notes are based on a course taught at MIT in [Fall 2022](#) and [Fall 2023](#). This is a live draft, and all parts will be updated regularly. Please send us an email if you find a mistake, typo, or missing reference.

Contents

1	Introduction	4
1.1	Decision Making	4
1.2	A Spectrum of Decision Making Problems	4
1.3	Minimax Perspective	6
1.4	Statistical Learning: Brief Refresher	7
1.5	Refresher: Random Variables and Averages	10
1.6	Online Learning and Prediction	12
1.6.1	Connection to Statistical Learning	14
1.6.2	The Exponential Weights Algorithm	15
1.7	Exercises	19
2	Multi-Armed Bandits	21
2.1	The Need for Exploration	23
2.2	The ϵ -Greedy Algorithm	24
2.3	The Upper Confidence Bound (UCB) Algorithm	27
2.4	Bayesian Bandits and the Posterior Sampling Algorithm*	30
2.5	Adversarial Bandits and the Exp3 Algorithm*	34
2.6	Deferred Proofs	35
2.7	Exercises	37
3	Contextual Bandits	38
3.1	Optimism: Generic Template	40
3.2	Optimism for Linear Models: The LinUCB Algorithm	43
3.3	Moving Beyond Linear Classes: Challenges	45
3.4	The ϵ -Greedy Algorithm for Contextual Bandits	46
3.5	Inverse Gap Weighting: An Optimal Algorithm for General Model Classes	49
3.5.1	Extending to Offline Regression	52
3.6	Exercises	53

4	Structured Bandits	55
4.1	Building Intuition: Optimism for Structured Bandits	58
4.1.1	UCB for Structured Bandits	58
4.1.2	The Eluder Dimension	59
4.1.3	Suboptimality of Optimism	62
4.2	The Decision-Estimation Coefficient	64
4.3	Decision-Estimation Coefficient: Examples	69
4.3.1	Cheating Code	69
4.3.2	Linear Bandits	70
4.3.3	Nonparametric Bandits	73
4.3.4	Further Examples	74
4.4	Relationship to Optimism and Posterior Sampling	75
4.4.1	Connection to Optimism	75
4.4.2	Connection to Posterior Sampling	77
4.5	Incorporating Contexts*	78
4.6	Additional Properties of the Decision-Estimation Coefficient*	79
4.7	Exercises	80
5	Reinforcement Learning: Basics	80
5.1	Finite-Horizon Episodic MDP Formulation	81
5.2	Planning via Dynamic Programming	82
5.3	Failure of Uniform Exploration	84
5.4	Analysis Tools	85
5.5	Optimism	87
5.6	The UCB-VI Algorithm for Tabular MDPs	88
5.6.1	Analysis for a Single Episode	90
5.6.2	Regret Analysis	92
6	General Decision Making	93
6.1	Setting	93
6.2	Refresher: Information-Theoretic Divergences	95
6.3	The Decision-Estimation Coefficient for General Decision Making	97
6.3.1	Basic Examples	98
6.4	E2D Algorithm for General Decision Making	100
6.4.1	Online Estimation with Hellinger Distance	102
6.5	Decision-Estimation Coefficient: Lower Bound on Regret	103
6.5.1	The Constrained Decision-Estimation Coefficient	104
6.5.2	Lower Bound	105
6.5.3	Proof of Proposition 28	106
6.5.4	Examples for the Lower Bound	110
6.6	Decision-Estimation Coefficient and E2D: Application to Tabular RL	112
6.6.1	Proof of Proposition 31	115
6.7	Tighter Regret Bounds for the Decision-Estimation Coefficient	118
6.7.1	Guarantees Based on Decision Space Complexity	118
6.7.2	General Divergences and Randomized Estimators	119
6.7.3	Optimistic Estimation	121
6.8	Decision-Estimation Coefficient: Structural Properties*	124
6.9	Deferred Proofs	125

6.10 Exercises	127
7 Reinforcement Learning: Function Approximation and Large State Spaces	129
7.1 Is Realizability Sufficient?	129
7.2 Linear Function Approximation	131
7.2.1 The LSVI-UCB Algorithm	132
7.2.2 Proof of Proposition 46	134
7.3 Bellman Rank	137
7.3.1 The BiLinUCB Algorithm	139
7.3.2 Proof of Proposition 47	141
7.3.3 Bellman Rank: Examples	145
7.3.4 Generalizations of Bellman Rank	147
7.3.5 Decision-Estimation Coefficient for Bellman Rank	148
A Technical Tools	156
A.1 Probabilistic Inequalities	156
A.1.1 Tail Bounds with Stopping Times	156
A.1.2 Tail Bounds for Martingales	156
A.2 Information Theory	158
A.2.1 Properties of Hellinger Distance	158
A.2.2 Change-of-Measure Inequalities	158
A.3 Minimax Theorem	159

1. INTRODUCTION

1.1 Decision Making

This is a course about learning to make decisions in an interactive, data-driven fashion. When we say interactive decision making, we are thinking of problems such as:

- Medical treatment: based on a patient’s medical history and vital signs, we need to decide what treatment will lead to the most positive outcome.
- Controlling a robot: based on sensor signals, we need to decide what signals to send to a robot’s actuators in order to navigate to a goal.

For both problems, we (the *learner/agent*) are interacting with an *unknown environment*. In the robotics example, we do not necessarily a-priori know how the signals we send to our robot’s actuators change its configuration, or what the landscape it’s trying to navigate looks like. However, because we are able to *actively* control the agent, we can *learn* to model the environment on the fly as we make decisions and collect data, which will reduce uncertainty and allow us to make better decisions in the future. The crux of the interactive decision making problem is to make decisions in a way that balances (i) exploring the environment to reduce our uncertainty and (ii) maximizing our overall performance (e.g., reaching a goal state as fast as possible).

Figure 1 depicts an idealized interactive decision making setting, which we will return to throughout this course. Here, at each round t , the agent (doctor) observes the medical history and vital signs of a patient, summarized in a *context* x^t , makes a treatment *decision* π^t , and then observes the outcomes of the treatment in the form of a *reward* r^t , and an auxiliary *observation* o^t about, say, illness progression. With time, we hope that the doctor will learn a good mapping $x^t \mapsto \pi^t$ from contexts to decisions. How can we develop an automated system that can achieve this goal?

It is tempting to cast the problem of finding a good mapping $x^t \mapsto \pi^t$ as a supervised learning problem. After all, modern deep neural networks are able to achieve excellent performance on many tasks, such as image classification and recognition, and it is not out of the question that there exists a good neural network for the medical example as well. The question is: how do we find it? In supervised learning, finding a good predictor often amounts to fitting an appropriate model—such as a neural network—to the data. In the above example, however, the available data may be limited to what treatments have been assigned to patients, potentially missing better options. It is the process of active data collection with a controlled amount of exploration that we would like to study in this course.

The decision making framework in Figure 1 generalizes many interactive decision making problems the reader might already be familiar with, including multi-armed bandits, contextual bandits, and reinforcement learning. We will cover the foundations of algorithm design and analysis for all of these settings from a unified perspective, with an emphasis on *sample efficiency* (i.e., how to learn a good decision making policy using as few rounds of interaction as possible).

1.2 A Spectrum of Decision Making Problems

To design algorithms for general interactive decision making problems such as Figure 1, there are many complementary challenges we must overcome. These challenges correspond

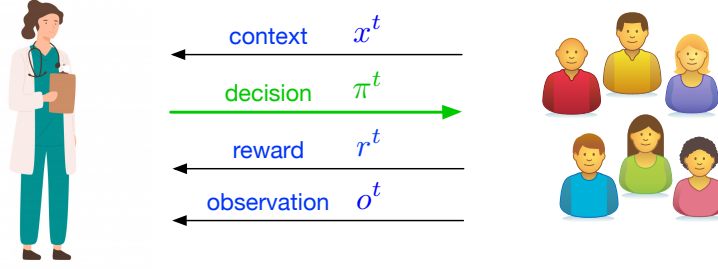


Figure 1: A general decision making problem.

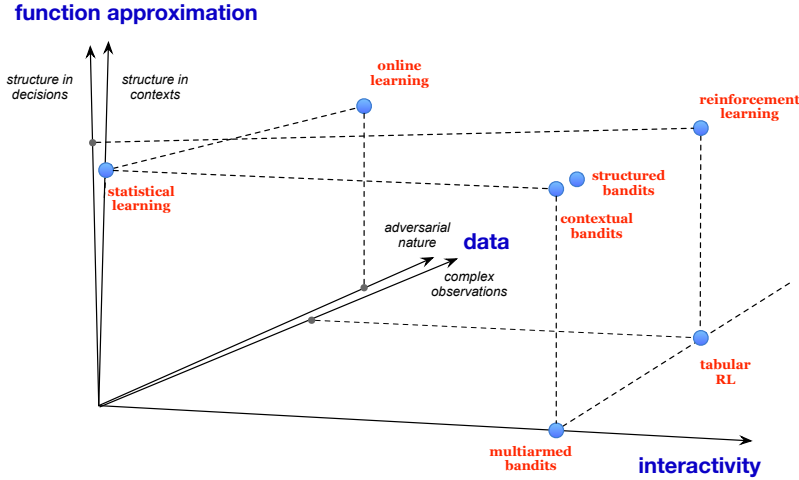


Figure 2: Landscape of decision making problems.

to different assumptions we can place on the underlying environment and decision making protocol, and give rise to what we describe as a *spectrum* of decision making problems, which is illustrated in Figure 2. There are three core challenges we will focus on throughout the course, which are given by the axes of Figure 2.

- **Interactivity.** Does the learning agent observe data passively, or do the decisions they make actively influence what data we collect? In the setting of Figure 1, the doctor observes the effects of the prescribed treatments, but not the counterfactuals (the effects of the treatments not given). Hence, doctor's decisions influence the data they can collect, which in turn may significantly alter the ability to estimate the effects of different treatments. On the other hand, in classical machine learning, a dataset is typically given to the learner upfront, with no control over how it is collected.
- **Function approximation and generalization.** In supervised statistical learning and estimation, one typically employs *function approximation* (e.g., models such as neural networks, kernels, or forests) to generalize across the space of covariates. For decision making, we can employ function approximation in a similar fashion, either to generalize across a space of contexts, or to generalize across the space of *decisions*. In the setting of Figure 1, the context x^t summarizing the medical history and vital

signs might be a highly structured object. Likewise, the treatment π^t might be a high-dimensional vector with interacting components, or a complex multi-stage treatment strategy.

- **Data.** Is the data (e.g., rewards or observations) observed by our learning algorithm produced by a fixed data-generating process, or does it evolve arbitrarily, and even adversarially in response to our actions? If there is fixed data-generating process, do we wish to directly model it, or should we instead aim to be agnostic? Do we observe only the labels of images, as in supervised learning, or a full trajectory of states/actions/rewards for a policy employed by the robot?

As shown in Figure 2, many basic decision making and learning frameworks (contextual bandits, structured bandits, statistical learning, online learning) can be thought of as idealized problems that each capture one or more of the possible challenges, while richer settings such as reinforcement learning encompass all of them.

Figure 2 can be viewed as a roadmap for the course. We start with a brief introduction to Statistical Learning (Section 1.4) and Online Learning (Section 1.6); the concepts and results stated here will serve as a backbone for the rest of the course. We will then study, in order, the problems of Multi-Armed Bandits (Section 2), Contextual Bandits (Section 3), Structured Bandits (Section 4), Tabular Reinforcement Learning (Section 5), General Decision Making (Section 6), and Reinforcement Learning with General Function Approximation (Section 7). Each of these topics will add a layer of complexity, and our aim is to develop a unified approach to all the aforementioned problems, both in terms of statistical complexity (the number of interactions required to achieve the goal), and in terms of algorithm design.

1.3 Minimax Perspective

For much of the course, we take a *minimax* point of view. Abstractly, let $\mathcal{M}$ be a set of possible models (or, choices for the environment) that can be encountered by the learner/decision maker. The set $\mathcal{M}$ can be thought of as representing the prior knowledge of the learner about the underlying environment. Let Alg denote a learning algorithm, and $\text{Perf}_T(\text{Alg}, M)$ be some notion of performance of algorithm Alg on model $M \in \mathcal{M}$ after T rounds of interaction (or—in passive learning—after observing T datapoints). We would like to develop algorithms that perform well, no matter what the model $M \in \mathcal{M}$ is, in the sense that Alg approximately solves the minimax problem

$$\min_{\text{Alg}} \max_{M \in \mathcal{M}} \text{Perf}_T(\text{Alg}, M). \quad (1.1)$$

$\nearrow \text{i.e. Regret, Loss}$

Understanding the *statistical complexity* (or, difficulty) of a given problem amounts to establishing matching (or nearly matching) upper bounds $\bar{\phi}_T(\mathcal{M})$ and lower bounds $\underline{\phi}_T(\mathcal{M})$ on the minimax value in (1.1). While developing such upper and lower bounds for specific model classes $\mathcal{M}$ of interest might be a simple task, the grand aim of this course is to develop a more fundamental, unified understanding of what makes *any* model class $\mathcal{M}$ easy versus hard, and to give sharp results for all (or nearly all) $\mathcal{M}$.

On the algorithmic side, we would like to better understand the scope of optimal algorithms that solve (1.1). While the minimax problem is itself an optimization problem, the space of all algorithms is typically prohibitively large. One of the key insights to be leveraged in this course is that for general decision making problems, we can restrict ourselves to algorithms that interleave a type of supervised learning called *online estimation* (this

will be described in [Sections 1.4 and 1.6](#)), with a principled choice of *exploration strategy* that balances greedily maximizing performance (exploitation) with information acquisition (exploration). As we show, such algorithms achieve or nearly achieve optimality in [\(1.1\)](#) for a surprisingly wide range of decision making problems.

1.4 Statistical Learning: Brief Refresher

We begin with a short refresher on the statistical learning problem. Statistical learning is a purely passive problem in which the learner does not directly interact with the environment, but it captures the challenge of *generalization and function approximation* in the context of [Figure 2](#).

In the statistical learning problem, we receive examples $(x^1, y^1), \dots, (x^T, y^T) \in \mathcal{X} \times \mathcal{Y}$, i.i.d. from a (unknown) distribution M^* . Here $x^t \in \mathcal{X}$ are *features* (sometimes called contexts or covariates), and $\mathcal{X}$ is the feature space. $y^t \in \mathcal{Y}$ are called *outcomes*, and $\mathcal{Y}$ is the outcome space. Given $(x^1, y^1), \dots, (x^T, y^T)$, the goal is to produce a model (or, estimator) $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}'$ that will do a good job *predicting* outcomes from features for future examples (x, y) drawn from M^* .¹

To measure prediction performance, we take as given a *loss function* $\ell : \mathcal{Y}' \times \mathcal{Y} \rightarrow \mathbb{R}$. Standard examples include:

- Regression, where common losses include the *square loss* $\ell(a, b) = (a - b)^2$ when $\mathcal{Y} = \mathcal{Y}' = \mathbb{R}$.
- Classification, where $\mathcal{Y} = \mathcal{Y}' = \{0, 1\}$ and we consider the indicator (or 0-1) loss $\ell(a, b) = \mathbb{I}\{a \neq b\}$.
- Conditional density estimation with the *logarithmic loss* (log loss). Here $\mathcal{Y}' = \Delta(\mathcal{Y})$, the set of distributions on $\mathcal{Y}$, and for $p \in \mathcal{Y}'$,

$$\ell_{\log}(p, y) = -\log p(y). \quad (1.2)$$

For a function $f : \mathcal{X} \rightarrow \mathcal{Y}'$, we measure the prediction performance via the *population* (or, “test”) loss:

$$L(f) := \mathbb{E}_{(x, y) \sim M^*}[\ell(f(x), y)]. \quad (1.3)$$

Letting $\mathcal{H}^T := \{(x^t, y^t)\}_{t=1}^T$ denote the dataset, a (deterministic) *algorithm* is a map that takes the dataset as input and returns a function/predictor:

$$\hat{f}(\cdot; \mathcal{H}^T) : \mathcal{X} \rightarrow \mathcal{Y}'. \quad (1.4)$$

The goal in designing algorithms is to ensure that $\mathbb{E}[L(\hat{f})]$ is minimized, where $\mathbb{E}[\cdot]$ denotes expectation with respect to the draw of the dataset $\mathcal{H}^T$. Without any assumptions, it is not possible to learn a good predictor unless the number of examples T scales with $|\mathcal{X}|$ (this is sometimes called the *no-free-lunch theorem*). The basic idea behind statistical learning is to work with a restricted class of functions

$$\mathcal{F} \subseteq \{f : \mathcal{X} \rightarrow \mathcal{Y}'\}$$

¹Note that we allow the outcome space $\mathcal{Y}$ to be different from the prediction space $\mathcal{Y}'$.

in order to facilitate generalization. The class $\mathcal{F}$ can be thought of as (implicitly) encoding prior knowledge about the structure of the data. For example, in computer vision, if the features x^t correspond to images and the outcomes y^t are labels (e.g., “cat” or “dog”), one might expect that choosing $\mathcal{F}$ to be a class of convolutional neural networks will work well, since this encodes spatial structure.

Remark 1 (Conditional density estimation): For the problem of conditional density estimation, we shall overload the notation and interchangeably write $f(x)$ and $f(\cdot|x)$ for the conditional distribution. In this setting, the learner is required to compute a distribution for each x rather than form a point estimate (see Figure 3). For an outcome y , the loss is the negative log of the conditional density for the outcome.

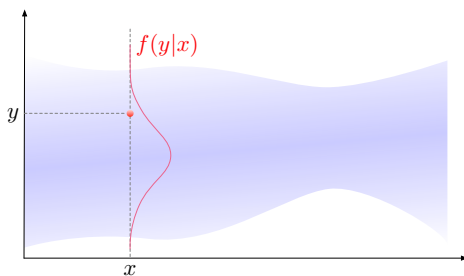


Figure 3: Conditional density estimation.

Empirical risk minimization and excess risk. The most basic and well-studied algorithmic principle for statistical learning is *Empirical Risk Minimization* (ERM). Define the empirical loss for the dataset $\mathcal{H}^T$ as

$$\widehat{L}(f) = \frac{1}{T} \sum_{i=1}^T \ell(f(x^i), y^i). \quad (1.5)$$

Then, the empirical risk minimizer with respect to the class $\mathcal{F}$ is given by

$$\widehat{f} \in \arg \min_{f \in \mathcal{F}} \widehat{L}(f). \quad (1.6)$$

To measure the performance of ERM and other algorithms that attempt to learn with $\mathcal{F}$, we consider *excess loss* (or, regret)

$$\mathcal{E}(f) = L(f) - \min_{f' \in \mathcal{F}} L(f'). \quad (1.7)$$

Intuitively, the quantity $\min_{f' \in \mathcal{F}} L(f')$ in (1.7) captures the best prediction performance any function in $\mathcal{F}$ can achieve, even with knowledge of the *true distribution*. If an algorithm $\widehat{f}$ has low excess risk, this means that we are predicting future outcomes nearly as well as any algorithm based on samples can hope to perform. ERM and other algorithms can ensure that $\mathcal{E}(\widehat{f})$ is small in expectation or with high probability over draw of the dataset $\mathcal{H}^T$.

Connection to estimation. An appealing feature of the formulation in (1.7) is that it does not presuppose any relationship between the class $\mathcal{F}$ and the data distribution; in other words, it is agnostic. However, if $\mathcal{F}$ does happen to be good at modeling the data distribution, the excess loss has an additional interpretation based on estimation.

Definition 1: For prediction with square loss, we say that the problem is *well-specified* (or, realizable) if the regression function $f^*(a) := \mathbb{E}[y|x = a]$ is in $\mathcal{F}$.

The regression function f^* can also be seen as a minimizer of $L(f)$ over measurable functions f , for the same reason that $\mathbb{E}_z(z - b)^2$ is minimized at $b = \mathbb{E}[z]$.

Lemma 1: For the square loss, if the problem is *well-specified*, then for all $f : \mathcal{X} \rightarrow \mathcal{Y}$,

$$\mathcal{E}(f) = \mathbb{E}_x[(f(x) - f^*(x))^2] \quad (1.8)$$

Proof of Lemma 1. Adding and subtracting f^* in the first term of (1.7), we have

$$\mathbb{E}(f(x) - y)^2 - \mathbb{E}(f^*(x) - y)^2 = \mathbb{E}(f(x) - f^*(x))^2 + 2 \mathbb{E}[(f^*(x) - y)(f(x) - f^*(x))].$$

$\begin{matrix} \nearrow^0 \\ \text{by def.} \end{matrix}$

□

Inspecting (1.8), we see that any f achieving low excess loss necessarily estimates the true regression function f^* ; hence, the goals of prediction and estimation coincide.

Guarantees for ERM. We give bounds on the excess loss of ERM for perhaps the simplest special case, in which $\mathcal{F}$ is finite.

Proposition 1: For any finite class $\mathcal{F}$, empirical risk minimization satisfies

$$\mathbb{E}[\mathcal{E}(\hat{f})] \lesssim \text{comp}(\mathcal{F}, T), \quad (1.9)$$

where

1. For any bounded loss (including classification), $\text{comp}(\mathcal{F}, T) = \sqrt{\frac{\log |\mathcal{F}|}{T}}$.
2. For square loss regression, if the problem is well-specified, $\text{comp}(\mathcal{F}, T) = \frac{\log |\mathcal{F}|}{T}$.

In addition, there exists a (different) algorithm that achieves $\text{comp}(\mathcal{F}, T) = \frac{\log |\mathcal{F}|}{T}$ for both square loss regression and conditional density estimation, even when the problem is not well-specified.

Henceforth, we shall use the symbol $\lesssim$ to indicate an inequality that holds up to constants, or other problem parameters deemed less important for the present discussion. As an example, the range of losses for the first part is hidden in this notation, and we only focus on the dependence of the right-hand side on $\mathcal{F}$ and T .

Recall Thm 4.1 in Ana's Lecture
 $L(\hat{h}) - L(h^*) \leq \sqrt{\frac{2(|\mathcal{H}|+1) \ln(2T/\delta)}{n}}$
 Pf: ① concentration inequality for fixed $h \in \mathcal{H}$
 ② union bound across the h
 ③ $L(\hat{h}) - L(h^*) \leq 2 \sup_{h \in \mathcal{H}} |L(h) - \hat{L}(h)|$

The rate $\text{comp}(\mathcal{F}, T) = \sqrt{\frac{\log |\mathcal{F}|}{T}}$ above is sometimes referred to as a *slow rate*, and is optimal for generic losses. The rate $\text{comp}(\mathcal{F}, T) = \frac{\log |\mathcal{F}|}{T}$ is referred to as a *fast rate*, and takes advantage of additional structure (curvature, or strong convexity) of the square loss. Critically, both bounds scale only with the cardinality of $\mathcal{F}$, and do not depend on the size of the feature space $\mathcal{X}$, which could be infinite. This reflects the fact that working with a restricted function class is allowing us to generalize across the feature space $\mathcal{X}$. In this context the cardinality $\log |\mathcal{F}|$ should be thought of a notion of *capacity*, or *expressiveness* for $\mathcal{F}$. Intuitively, choosing a larger, more expressive class will require a larger amount of data, but will make the excess loss bound in (1.7) more meaningful, since the benchmark will be stronger.

$$\min_{f \in \mathcal{F}} \mathcal{L}(f)$$

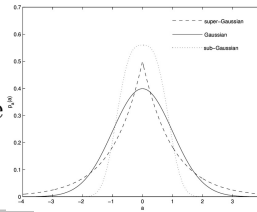
Remark 2 (From finite to infinite classes): Throughout these lecture notes, we restrict our attention to finite classes whenever possible in order to simplify presentation. If one wishes to move beyond finite classes, a well-developed literature within statistical learning provides various notions of complexity for $\mathcal{F}$ that lead to bounds on $\text{comp}(\mathcal{F}, T)$ for ERM and other algorithms. These include the Vapnik-Chervonenkis (VC) dimension for classification, Rademacher complexity, and covering numbers. Standard references include Bousquet et al. [20], Boucheron et al. [19], Anthony and Bartlett [10], Shalev-Shwartz and Ben-David [77].

1.5 Refresher: Random Variables and Averages

To prove Proposition 1 and similar generalization bounds, the main tools we will use are *concentration inequalities* (or, tail bounds) for random variables.

Definition 2: A random variable Z is sub-Gaussian with variance factor (or variance proxy) σ^2 if

$$\forall \eta \in \mathbb{R}, \quad \mathbb{E} e^{\eta(Z - \mathbb{E}[Z])} \leq e^{\sigma^2 \eta^2 / 2}.$$



Note that if $Z \sim \mathcal{N}(0, \sigma^2)$ is *Gaussian* with variance σ^2 , then it is sub-Gaussian with variance proxy σ^2 . In this sense, sub-Gaussian random variables generalize the tail behavior of Gaussians. A standard application of Chernoff method yields the following result.

Lemma 2: If $Z_1, \dots, Z_T$ are i.i.d. sub-Gaussian random variables with variance proxy σ^2 , then

$$\mathbb{P} \left(\frac{1}{T} \sum_{i=1}^T Z_i - \mathbb{E}[Z] \geq u \right) \leq \exp \left\{ -\frac{Tu^2}{2\sigma^2} \right\} \quad (1.10)$$

Proof by Markov inequality
Sum of sub-Gaussian random variables is sub-Gaussian

Applying this result with Z and $-Z$ and taking a union bound yields the following two-sided guarantee:

$$\mathbb{P} \left(\left| \frac{1}{T} \sum_{i=1}^T Z_i - \mathbb{E}[Z] \right| \geq u \right) \leq 2 \exp \left\{ -\frac{Tu^2}{2\sigma^2} \right\}. \quad (1.11)$$

Setting the right-hand side of (1.11) to δ and solving for u , we find that for any $\delta \in (0, 1)$, with probability at least $1 - \delta$,

$$\left| \frac{1}{T} \sum_{i=1}^T Z_i - \mathbb{E}[Z] \right| \leq \sqrt{\frac{2\sigma^2 \log(2/\delta)}{T}}. \quad (1.12)$$

Remark 3 (Union bound): The factor 2 under the logarithm in (1.12) is the result of applying union bound to (1.10). Throughout the course, we will frequently apply the union bound to multiple—say N —high probability events involving sub-Gaussian random variables. In this case, the union bound will result in terms of the form $\log(N/\delta)$. The mild logarithmic dependence is due to the sub-Gaussian tail behavior of the averages.

The following result shows that any bounded random variable is sub-Gaussian.

Lemma 3 (Hoeffding’s inequality): Any random variable Z taking values in $[a, b]$ is sub-Gaussian with variance proxy $(b - a)^2/4$, i.e.

$$\forall \eta \in \mathbb{R}, \quad \ln \mathbb{E} \exp\{-\eta(Z - \mathbb{E}[Z])\} \leq \frac{\eta^2(b - a)^2}{8}. \quad (1.13)$$

As a consequence, for i.i.d. random variables $Z_1, \dots, Z_T$ taking values in $[a, b]$ almost surely, with probability at least $1 - \delta$,

$$\frac{1}{T} \sum_{i=1}^T Z_i - \mathbb{E}[Z] \leq (b - a) \sqrt{\frac{\log(1/\delta)}{2T}} \quad (1.14)$$

In particular, in the setting of Section 1.4,

Using Hoeffding’s inequality, we can prove now prove Part 1 (the slow rate) from Proposition 1.

sample mean will not be much farther from the population mean.

Lemma 4 (Proposition 1, Part 1): Let $\mathcal{F} = \{f : \mathcal{X} \rightarrow \mathcal{Y}\}$ be finite, and assume $\ell \circ f \in [0, 1]$ almost surely. Then with probability at least $1 - \delta$, ERM satisfies

$$L(\hat{f}) - \min_{f \in \mathcal{F}} L(f) \leq 2 \sqrt{\frac{\log(2|\mathcal{F}|/\delta)}{2T}}.$$

Proof of Lemma 4. For any $f \in \mathcal{F}$, we can write

$$L(\hat{f}) - L(f) = \left[L(\hat{f}) - \hat{L}(\hat{f}) \right] + \left[\hat{L}(\hat{f}) - \hat{L}(f) \right] + \left[\hat{L}(f) - L(f) \right].$$

Observe that for all $f : \mathcal{X} \rightarrow \mathcal{Y}$, we have

$$\left| L(f) - \hat{L}(f) \right| = \left| \mathbb{E} \ell(f(X), Y) - \frac{1}{T} \sum_{i=1}^T \ell(f(X_i), Y_i) \right|.$$

By union bound and [Lemma 3](#), with probability at least $1 - |\mathcal{F}|\delta$,

$$\forall f \in \mathcal{F}, \quad \left| \mathbb{E} \ell(f(X), Y) - \frac{1}{T} \sum_{i=1}^T \ell(f(X_i), Y_i) \right| \leq \sqrt{\frac{\log(2/\delta)}{2T}} \quad (1.15)$$

□

To deduce the in-expectation bound of [Proposition 1](#) from the high-probability tail bound of [Lemma 4](#), a standard technique of “integrating out the tail” is employed. More precisely, for a nonnegative random variable U , it holds that $\mathbb{E}[U] \leq \tau + \int_{\tau}^{\infty} \mathbb{P}(U \geq z) dz$ for all $\tau > 0$; choosing $\tau \propto T^{-1/2}$ concludes the proof.

To prove the Part 2 (the fast rate) from [Proposition 1](#), we need a more refined concentration inequality (Bernstein’s inequality), which gives tighter guarantees for random variables with small variance.

Lemma 5 (Bernstein’s inequality): Let $Z_1, \dots, Z_T, Z$ be i.i.d. with variance $\mathbb{V}(Z_i) = \sigma^2$, and range $|Z - \mathbb{E} Z| \leq B$ almost surely. Then with probability at least $1 - \delta$,

$$\frac{1}{T} \sum_{i=1}^T Z_i - \mathbb{E} Z \leq \sigma \sqrt{\frac{2 \log(1/\delta)}{T}} + \frac{B \log(1/\delta)}{3T}. \quad (1.16)$$

The proof for Part 2 is given as an exercise in [Section 1.7](#). We refer the reader to [Appendix A.1](#) for further background on tail bounds.

1.6 Online Learning and Prediction

We now move on to the problem of *online learning*, or sequential prediction. The online learning problem generalizes statistical learning on two fronts:

- Rather than receiving a batch dataset of T examples all at once, we receive the examples (x^t, y^t) one by one, and must predict y^t from x^t only using the examples we have already observed.
- Instead of assuming that examples are drawn from a fixed distribution, we allow examples to be generated in an arbitrary, potentially adversarial fashion.

Online Learning Protocol

for $t = 1, \dots, T$ **do**

 Compute predictor $\hat{f}^t : \mathcal{X} \rightarrow \mathcal{Y}$

 Observe $(x^t, y^t) \in \mathcal{X} \times \mathcal{Y}$

In more detail, at each timestep t , given the examples

$$\mathcal{H}^{t-1} = \{(x^1, y^1), \dots, (x^{t-1}, y^{t-1})\} \quad (1.17)$$

observed so far, the algorithm produces a predictor

$$\hat{f}^t = \hat{f}^t(\cdot \mid \mathcal{H}^{t-1}),$$

which aims to predict the outcome y^t from the features x^t . The algorithm's goal is to minimize the cumulative loss over T rounds, given by

$$\sum_{t=1}^T \ell(\hat{f}^t(x^t), y^t)$$

for a known loss function $\ell : \mathcal{Y}' \times \mathcal{Y} \rightarrow \mathbb{R}$; the cumulative loss can be thought of as a sum of “out-of-sample” prediction errors. Since we will not be placing assumptions on the data-generating process, it is not possible to make meaningful statements about the cumulative loss itself. However, we can aim to ensure that this cumulative loss is not much worse than the best empirical explanation of the data by functions in a given class $\mathcal{F}$. That is, we measure the algorithm's performance via *regret* to $\mathcal{F}$:

$$\mathbf{Reg} = \sum_{t=1}^T \ell(\hat{f}^t(x^t), y^t) - \min_{f \in \mathcal{F}} \sum_{t=1}^T \ell(f(x^t), y^t). \quad (1.18)$$

Our aim is to design prediction algorithms that keep regret small for *any* sequence of data. As in statistical learning, the class $\mathcal{F}$ should be thought of as capturing our prior knowledge about the problem, and might be a linear model or neural network. At first glance, keeping the regret small for arbitrary sequences might seem like an impossible task, as it stands in stark contrast with statistical learning, where data is generated i.i.d. from a fixed distribution. Nonetheless, we will that algorithms with guarantees similar to those for statistical learning are available.

Let us remark that it is often useful to apply online learning methods in settings where data is not fully adversarial, but evolves according to processes too difficult to directly model. For example, in the chapters that follow, we will apply online methods as a sub-routine with more sophisticated algorithms for decision making. Here, the choice of past decisions, while in our purview, does not look like i.i.d. or simple time-series data.

Remark 4 (Proper learning, improper learning, and randomization): The online learning protocol does not require that $\hat{f}^t$ lies in $\mathcal{F}$ ($\hat{f}^t \in \mathcal{F}$). A method that chooses functions from $\mathcal{F}$ will be called *proper*, and the one that selects predictors outside of $\mathcal{F}$ will be called *improper*. It will also be useful to allow for *randomized* predictions of the form

$$\hat{f}^t \sim q^t(\cdot | \mathcal{H}^{t-1}),$$

where q^t is a distribution on functions, typically on elements of $\mathcal{F}$. For randomized predictions, we slightly abuse notation and write regret as

$$\mathbf{Reg} = \sum_{i=1}^T \mathbb{E}_{\hat{f}^i \sim q^i} [\ell(\hat{f}^i(x^i), y^i)] - \min_{f \in \mathcal{F}} \sum_{i=1}^T \ell(f(x^i), y^i). \quad (1.19)$$

The algorithms we introduce in the sequel below ensure small regret even if data are adversarially and adaptively chosen. More precisely, for deterministic algorithms, (x^t, y^t) may be chosen based on $\hat{f}^t$ and all the past data, while for randomized algorithms, Nature can only base this choice on q^t .

In the context of [Figure 2](#), online learning generalizes statistical learning by considering arbitrary sequences of data, but still allows for general-purpose function approximation and generalization via the class $\mathcal{F}$. While the setting involves making predictions in an online fashion, we do not think of this as an *interactive* decision making problem, because the predictions made by the learning agent do not directly influence what data the agent gets to observe.

1.6.1 Connection to Statistical Learning

Online learning can be thought of as a generalization of statistical learning, and in fact, algorithms for online learning immediately yield algorithms for statistical learning via a technique called *online-to-batch conversion*. This result, which is formalized by the following proposition, rests on two observations: the cumulative loss of the algorithm looks like a sum of out-of-sample errors, and the minimum empirical fit to realized data (over $\mathcal{F}$) is, on average, a harder (that is, smaller) benchmark than the minimum expected loss in $\mathcal{F}$.

Proposition 2: Suppose the examples $(x^1, y^1), \dots, (x^T, y^T)$ are drawn i.i.d. from a distribution M^* , and suppose the loss function $a \mapsto \ell(a, b)$ is convex in the first argument for all b . Then for any online learning algorithm, if we define

$$\hat{f}(x) = \frac{1}{T} \sum_{t=1}^T \hat{f}^t(x),$$

we have

$$\mathbb{E}[\mathcal{E}(\hat{f})] \leq \frac{1}{T} \cdot \mathbb{E}[\mathbf{Reg}].$$

Proof of Proposition 2. Let $(x, y) \sim M^*$ be a fresh sample which is independent of the history $\mathcal{H}^T$. First, by Jensen's inequality,

$$\mathbb{E}[L(\hat{f})] = \mathbb{E}\left[\mathbb{E}_{(x,y)} \ell\left(\frac{1}{T} \sum_{t=1}^T \hat{f}^t(x), y\right)\right] \leq \mathbb{E}\left[\frac{1}{T} \sum_{t=1}^T \mathbb{E}_{(x,y)} \ell(\hat{f}^t(x), y)\right] \quad (1.20)$$

which is equal to

$$\mathbb{E}\left[\frac{1}{T} \sum_{t=1}^T \mathbb{E}_{(x^t, y^t)} \ell(\hat{f}^t(x^t), y^t)\right] \quad (1.21)$$

since $\hat{f}^t$ is a function of $\mathcal{H}^{t-1}$ and (x, y) and (x^t, y^t) are i.i.d. Second,

$$\min_{f \in \mathcal{F}} L(f) = \min_{f \in \mathcal{F}} \mathbb{E}\left[\frac{1}{T} \sum_{t=1}^T \ell(f(x^t), y^t)\right] \geq \mathbb{E}\left[\min_{f \in \mathcal{F}} \frac{1}{T} \sum_{t=1}^T \ell(f(x^t), y^t)\right] \quad (1.22)$$

□

In light of [Proposition 2](#), one can interpret regret as generalizing the notion of excess risk from i.i.d. data to arbitrary sequences.

Similar to Lemma 1 in the setting of statistical learning, the regret for online learning has an additional interpretation in terms of *estimation* if the outcomes for the problem are well-specified.

Lemma 6: Suppose that the features $x^1, \dots, x^T$ are generated in an arbitrary fashion, but that for all t , the variable y^t is random with mean given by a fixed function $f^* \in \mathcal{F}$:

$$\mathbb{E}[y^t \mid x^t = x] = f^*(x).$$

Then for the problem of prediction with square loss,

$$\mathbb{E}[\mathbf{Reg}] \geq \mathbb{E} \left[\sum_{t=1}^T (\hat{f}^t(x^t) - f^*(x^t))^2 \right].$$

Notably, this result holds even if the features $x^1, \dots, x^T$ are generated adversarially, with no prior knowledge of the sequence. This is a significant departure from classical estimation results in statistics, where estimation of an unknown function is typically done over a fixed, known sequence (“design”) $x^1, \dots, x^T$, or with respect to an i.i.d. dataset.

1.6.2 The Exponential Weights Algorithm

The main online learning algorithm is the *Exponential Weights* algorithm, which is applicable to finite classes $\mathcal{F}$. At each time t , the algorithm computes a distribution $q^t \in \Delta(\mathcal{F})$ via

$$q^t(f) \propto \exp \left\{ -\eta \sum_{i=1}^{t-1} \ell(f(x^i), y^i) \right\}, \quad (1.23)$$

where $\eta > 0$ is a learning rate. Based on q^t , the algorithm forms the prediction $\hat{f}^t$. We give two variants of the method here.

Exponential Weights (averaged)
for $t = 1, \dots, T$ **do**
 Compute q^t in (1.23).
 Let $\hat{f}^t = \mathbb{E}_{f \sim q^t}[f]$.
 Observe (x^t, y^t) , incur $\ell(\hat{f}^t(x^t), y^t)$.

Exponential Weights (randomized)
for $t = 1, \dots, T$ **do**
 Compute q^t in (1.23).
 Sample $\hat{f}^t \sim q^t$.
 Observe (x^t, y^t) , incur $\ell(\hat{f}^t(x^t), y^t)$.

The only difference between these variants lies in whether we compute the prediction $\hat{f}^t$ from q^t via

$$\hat{f}^t = \mathbb{E}_{f \sim q^t}[f], \quad \text{or} \quad \hat{f}^t \sim q^t. \quad (1.24)$$

The latter can be applied to any bounded loss functions, while the former leads to faster rates for specific losses such as the square loss and log loss, but is only applicable when $\mathcal{Y}$ is convex. Note that the averaged version is inherently improper, while the second is proper, yet randomized. From the point of view of regret, the key difference between these two versions is the placement of “ $\mathbb{E}_{f \sim q^t}$ ”: For the averaged version it is inside the loss function, and for the randomized version it is outside (see (1.19)). The averaged version can therefore take advantage of the structure of the loss function, such as strong convexity, leading to

faster rates. The following result shows that Exponential Weights leads to regret bounds for online learning, with rates that parallel those in [Proposition 1](#).

Proposition 3: For any finite class $\mathcal{F}$, the Exponential Weights algorithm (with appropriate choice of η) satisfies

$$\frac{1}{T} \mathbf{Reg} \lesssim \text{comp}(\mathcal{F}, T) \quad (1.25)$$

for any sequence, where:

1. For arbitrary bounded losses (including classification), $\text{comp}(\mathcal{F}, T) = \sqrt{\frac{\log |\mathcal{F}|}{T}}$. This is achieved by the randomized variant.
2. For regression with the square loss and conditional density estimation with the log loss, $\text{comp}(\mathcal{F}, T) = \frac{\log |\mathcal{F}|}{T}$. This is achieved by the averaged variant.

We now turn to the proof of [Proposition 3](#). Since we are not placing any assumptions on the data generating process, we cannot hope to control the algorithm's loss at any particular time t , but only cumulatively. It is then natural to employ amortized analysis with a potential function.

In more detail, the proof of [Proposition 3](#) relies on several steps, common to standard analyses of online learning: (i) define a potential function, (ii) relate the increase in potential at each time step, to the loss of the algorithm, (iii) relate cumulative loss of any expert $f \in \mathcal{F}$ to the final potential. For the Exponential Weights Algorithm, the proof relies on the following potential for time t , parameterized by $\eta > 0$:

$$\Phi_\eta^t = -\log \sum_{f \in \mathcal{F}} \exp \left\{ -\eta \sum_{i=1}^t \ell(f(x^i), y^i) \right\}. \quad (1.26)$$

The choice of this potential is rather opaque, and a full explanation of its origin is beyond the scope of the course, but we mention in passing that there are principled ways of coming up with potentials in general online learning problems.

Proof of Proposition 3. We first prove the second statement, focusing on conditional density with the logarithmic loss; for the square loss, see [Remark 6](#) below.

Proof for Part 2: Log loss. Recall that for each x , $f(x)$ is a distribution over $\mathcal{Y}$, and $\ell_{\log}(f(x), y) = -\log f(y|x)$ where we abuse the notation and write $f(x)$ and $f(\cdot|x)$ interchangeably. With $\eta = 1$, the averaged variant of exponential weights satisfies

$$\hat{f}^t(y^t|x^t) = \sum_{f \in \mathcal{F}} q^t(f) f(y^t|x^t) = \sum_{f \in \mathcal{F}} f(y^t|x^t) \frac{\exp \left\{ -\sum_{i=1}^{t-1} \ell_{\log}(f(x^i), y^i) \right\}}{\sum_{f \in \mathcal{F}} \exp \left\{ -\sum_{i=1}^{t-1} \ell_{\log}(f(x^i), y^i) \right\}}, \quad (1.27)$$

and thus

$$\ell_{\log}(\hat{f}^t(x^t), y^t) = -\log \hat{f}^t(y^t|x^t) = \Phi_1^t - \Phi_1^{t-1}. \quad (1.28)$$

$$\begin{aligned} \Phi_1^+ &= -\log \sum_f \exp \left\{ -\sum_{i=1}^{t-1} \ell_{\log}(f(x^i), y^i) \right\} \cdot \exp \left\{ -\ell_{\log}(f(x^t), y^t) \right\} \\ &= -\log \sum_f \exp \left\{ -\sum_{i=1}^{t-1} \ell_{\log}(f(x^i), y^i) \right\} \cdot f(y^t|x^t) \end{aligned}$$

Hence, by telescoping

$$\sum_{t=1}^T \ell_{\log}(\hat{f}(x^t), y^t) = \Phi_1^T - \Phi_1^0. \quad \Phi_1^0 = -\log \sum_f 1 = -\log |\mathcal{F}|$$

Finally, observe that $\Phi_1^0 = -\log |\mathcal{F}|$ and, since $-\log$ is monotonically decreasing, we have

$$\Phi_1^T \leq -\log \exp \left\{ -\sum_{i=1}^T \ell_{\log}(f^*(x^i), y^i) \right\} = \sum_{i=1}^T \ell_{\log}(f^*(x^i), y^i), \quad (1.29)$$

for any $f^* \in \mathcal{F}$. This establishes the result for conditional density estimation with the log loss. As already discussed, the above proof follows the strategy: the loss on each round related to change in potential (1.28), and the cumulative loss of any expert is related to the final potential (1.29). We now aim to replicate these steps for arbitrary bounded losses.

Proof for Part 1: Generic loss. To prove this result, we build on the log loss result above. First, observe that without loss of generality, we may assume that $\ell \circ f \in [0, 1]$ for all $f \in \mathcal{F}$ and (x, y) , as we can always re-scale the problem. The randomized variant of exponential weights (1.24) satisfies

$$\mathbb{E}_{\hat{f}^t \sim q^t}[\ell(\hat{f}^t(x^t), y^t)] = \sum_{f \in \mathcal{F}} \ell(f(x^t), y^t) \frac{\exp \left\{ -\eta \sum_{i=1}^{t-1} \ell(f(x^i), y^i) \right\}}{\sum_{f \in \mathcal{F}} \exp \left\{ -\eta \sum_{i=1}^{t-1} \ell(f(x^i), y^i) \right\}}. \quad (1.30)$$

Hoeffding's inequality (1.13) implies that $\forall \epsilon \in (0, 1], \mathbb{P} \left(\left| \frac{1}{t} \sum_{i=1}^t \ell(f(x^i), y^i) - \mathbb{E}[\ell(f(x^t), y^t)] \right| \geq \epsilon \right) \leq \exp(-t\epsilon^2)$

$$\eta \mathbb{E}_{\hat{f}^t \sim q^t}[\ell(\hat{f}^t(x^t), y^t)] \leq -\log \sum_{f \in \mathcal{F}} \frac{\exp \left\{ -\eta \ell(f(x^t), y^t) \right\} \exp \left\{ -\eta \sum_{i=1}^{t-1} \ell(f(x^i), y^i) \right\}}{\sum_{f \in \mathcal{F}} \exp \left\{ -\eta \sum_{i=1}^{t-1} \ell(f(x^i), y^i) \right\}} + \frac{\eta^2}{8}. \quad (1.31)$$

Note that the right-hand side of this inequality is simply

$$\Phi_\eta^t - \Phi_\eta^{t-1} + \frac{\eta^2}{8},$$

establishing the analogue of (1.28). Summing over t , this gives

$$\eta \sum_{t=1}^T \mathbb{E}_{\hat{f}^t \sim q^t}[\ell(\hat{f}^t(x^t), y^t)] \leq \Phi_\eta^T - \Phi_\eta^0 + \frac{T\eta^2}{8}. \quad (1.32)$$

As in the first part, for any $f^* \in \mathcal{F}$, we can upper bound

$$\Phi_\eta^T \leq \eta \sum_{t=1}^T \ell(f^*(x^t), y^t),$$

while $\Phi_\eta^0 = -\log |\mathcal{F}|$. Hence, we have that for any $f^* \in \mathcal{F}$,

$$\sum_{t=1}^T \mathbb{E}_{\hat{f}^t \sim q^t}[\ell(\hat{f}^t(x^t), y^t)] - \ell(f^*(x^t), y^t) \leq \frac{T\eta}{8} + \frac{\log |\mathcal{F}|}{\eta}.$$

With $\eta = \sqrt{\frac{8 \log |\mathcal{F}|}{T}}$, we conclude that

$$\sum_{t=1}^T \mathbb{E}_{\hat{f}^t \sim q^t} [\ell(\hat{f}^t(x^t), y^t)] - \ell(f^*(x^t), y^t) \leq \sqrt{\frac{T \log |\mathcal{F}|}{2}}. \quad (1.33)$$

□

Observe that Hoeffding's inequality was all that was needed for [Lemma 4](#). Curiously enough, it was also the only nontrivial step in the proof of [Proposition 3](#). In fact, the connection between probabilistic inequalities and online learning regret inequalities (that hold for arbitrary sequences) runs much deeper.

Remark 5 (Beyond finite classes): As in statistical learning, there are (sequential) complexity measures for $\mathcal{F}$ that can be used to generalize the regret bounds in [Proposition 3](#) to infinite classes. In general, the optimal regret for a class $\mathcal{F}$ will reflect the statistical capacity of the class [\[69\]](#).

Remark 6 (Mixable losses): We did not provide a proof of [Proposition 3](#) for square loss. It is tempting to reduce square loss regression to density estimation by taking the conditional density to be a Gaussian distribution. Indeed, the log loss of a distribution with density proportional to $\exp\{-(\hat{f}^t(x^t) - y^t)^2\}$ is, up to constants, the desired square loss. However, the mixture in [\(1.27\)](#) does not immediately lead to a prediction strategy for the square loss, as the expectation appears in the wrong location. This issue is fixed by a notion known as *mixability*.

We say that a loss ℓ is *mixable* with parameter η if there exists a constant $c > 0$ such that the following holds: for any x and a distribution $q \in \Delta(\mathcal{F})$, there exists a prediction $\hat{f}(x) \in \mathcal{Y}'$ such that for all $y \in \mathcal{Y}$,

$$\ell(\hat{f}(x), y) \leq -\frac{c}{\eta} \log \left(\sum_{f \in \mathcal{F}} q(f) \exp\{-\eta \ell(f(x), y)\} \right). \quad (1.34)$$

If loss is mixable, then given the exponential weights distribution q^t , the best prediction $\hat{y}^t = \hat{f}^t(x^t)$ can be written (by bringing the right-hand side of [\(1.34\)](#) to the left side) as an optimization problem

$$\arg \min_{\hat{y}^t \in \mathcal{Y}'} \max_{y^t \in \mathcal{Y}} \left[\ell(\hat{y}^t, y^t) + \frac{c}{\eta} \log \left(\sum_{f \in \mathcal{F}} q^t(f) \exp\{-\eta \ell(f(x^t), y^t)\} \right) \right] \quad (1.35)$$

which is equivalent to

$$\arg \min_{\hat{y}^t \in \mathcal{Y}'} \max_{y^t \in \mathcal{Y}} \left[\ell(\hat{y}^t, y^t) + \frac{c}{\eta} \log \left(\sum_{f \in \mathcal{F}} \exp\left\{-\eta \sum_{i=1}^t \ell(f(x^i), y^i)\right\} \right) \right] \quad (1.36)$$

$$\log \log: -\log \sum_f g(f) f(y|x)$$

once we remove the normalization factor. With this choice, mixability allows one to replicate the proof of [Proposition 3](#) for the logarithmic loss, with the only difference being that (1.27) (after applying $-\log$ to both sides) becomes an inequality. It can be verified that square loss is mixable with parameter $\eta = 2$ and $c = 1$ when $\mathcal{Y} = \mathcal{Y}' = [0, 1]$, leading to the desired fast rate for square loss in [Proposition 3](#). The idea of translating the English statement “there exists a strategy such that for any outcome...” into a min-max inequality will come up again in the course.

Remark 7 (Online linear optimization): For the slow rate in [Proposition 3](#), the nature of the loss and the dependence on the function f is immaterial for the proof. The guarantee can be stated in a more abstract form that depends only on the vector of losses for functions in $\mathcal{F}$ as follows. Let $|\mathcal{F}| = N$. For timestep t , define $\ell_f^t = \ell(f(x^t), y^t)$ and $\ell^t = (\ell_{f_1}^t, \dots, \ell_{f_N}^t) \in \mathbb{R}^N$ for $\mathcal{F} = \{f_1, \dots, f_N\}$. For a randomized strategy $q^t \in \Delta([N])$, expected loss of the learner can be written as

$$\mathbb{E}_{\hat{f}^t \sim q^t}[\ell(\hat{f}^t(x^t), y^t)] = \langle q^t, \ell^t \rangle,$$

and the expected regret can be written as

$$\mathbf{Reg} = \sum_{t=1}^T \langle q^t, \ell^t \rangle - \min_{j \in \{1, \dots, N\}} \sum_{t=1}^T \langle e_j, \ell^t \rangle \quad (1.37)$$

where $e_j \in \mathbb{R}^N$ is the standard basis vector with 1 in j th position. In its most general form, the exponential weights algorithm gives bounds on the regret in (1.37) for any sequence of vectors $\ell^1, \dots, \ell^T$, and the update takes the form

$$q^t(k) \propto \exp \left\{ -\eta \sum_{i=1}^{t-1} \ell^i(k) \right\}.$$

This formulation can be viewed as a special case of a problem known as *online linear optimization*, and the exponential weights method can be viewed as an instance of an algorithm known as *mirror descent*.

1.7 Exercises

Exercise 1 (Proposition 1, Part 2.): Consider the setting of [Proposition 1](#), where $(x^1, y^1), \dots, (x^T, y^T)$ are i.i.d., $\mathcal{F} = \{f : \mathcal{X} \rightarrow [0, 1]\}$ is finite, the true regression function satisfies $f^* \in \mathcal{F}$, and $Y_i \in [0, 1]$ almost surely. Prove that empirical risk minimizer $\hat{f}$ with respect to square loss satisfies the following bound on excess risk. With probability at least $1 - \delta$,

$$\mathcal{E}(\hat{f}) \lesssim \frac{\log(|\mathcal{F}|/\delta)}{T}. \quad (1.38)$$

Follow these steps: